

检查点 2 实验报告

数据清洗、分块打包与加载性能对比

2026 年 4 月 17 日

目录

1 实验目标	2
2 实验环境	2
2.1 硬件环境	2
2.2 软件环境	2
3 数据集说明	2
3.1 VOC2012	2
3.2 COCO2017	3
4 统一格式整理	3
4.1 统一目标	3
4.2 统一脚本	3
4.3 统一结果	4
5 实验设计	4
5.1 打包格式	4
5.2 加载方式	4
5.3 Dataset 与 DataLoader	4
5.4 批大小设置	4
5.5 控制变量	5
6 Stage1: 加载器参数扫描	5
6.1 目的	5
6.2 结论	5
6.3 结果图	6
6.4 关键结果	8

7	Stage2: 格式与分块大小对比	8
7.1	实验设置	8
7.2	结果图	8
7.3	关键结果	10
8	Stage3: shuffle 对照实验	10
8.1	实验目的	10
8.2	实验设置	10
8.3	结果图	11
8.4	关键结果	13
9	结论	13
10	附录: 关键实现文件	14

1 实验目标

本次实验围绕检查点 2 的要求展开，目标包括：

1. 将 VOC2012 与 COCO2017 整理成统一的数据格式。
2. 基于统一格式，比较原始加载与分块加载的性能差异。
3. 比较至少 6 种打包方式对加载速度的影响。
4. 比较不同分块大小对加载吞吐的影响。
5. 找到当前实验条件下最快的数据加载方式。

2 实验环境

2.1 硬件环境

- GPU: $8 \times$ NVIDIA GeForce RTX 4090
- 驱动版本: 590.48.01
- CUDA 版本: 13.1

2.2 软件环境

- Conda 环境: yolov1
- Python: 3.10.20
- PyTorch: 2.11.0+cu130
- TorchVision: 0.26.0+cu130

3 数据集说明

3.1 VOC2012

整理后正式使用目录：

- 图片: VOCdevkit/VOC2012/JPEGImages
- 标注: VOCdevkit/VOC2012/Annotations
- 划分: VOCdevkit/VOC2012/ImageSets/Main/train.txt、val.txt

3.2 COCO2017

整理后正式使用目录:

- 图片: train2017、val2017
- 标注: annotations/instances_train2017.json、instances_val2017.json

4 统一格式整理

4.1 统一目标

为避免原始 XML 与 JSON 解析方式差异干扰加载对比实验, 先将两个数据集统一为逐样本记录格式。每条记录至少包含:

- sample_id
- dataset_source
- split
- image_id
- image_path
- width、height
- boxes
- labels
- label_names
- original_category_ids
- iscrowd

4.2 统一脚本

统一脚本为:

- build_unified_detection_manifest.py

输出内容包括:

- manifests/*.jsonl
- metadata/schema.json
- metadata/class_maps.json
- metadata/summary.json

4.3 统一结果

- voc2012_train: 5717 images, 15774 boxes
- voc2012_val: 5823 images, 15787 boxes
- coco2017_train: 118287 images, 860001 boxes
- coco2017_val: 5000 images, 36781 boxes
- all: 134827 images, 928343 boxes

5 实验设计

5.1 打包格式

本次实验比较 6 种分块打包方式：

1. pickle
2. npz
3. pt
4. safetensors
5. hdf5
6. msgpack

5.2 加载方式

比较两种主路径：

1. 不分块，直接基于原始统一 manifest 加载图片
2. 分块打包后，通过分块索引加载数据

5.3 Dataset 与 DataLoader

所有实验均通过 PyTorch 的 Dataset 与 DataLoader 完成：

- 原始加载：RawManifestDataset
- 分块加载：ChunkedPackedDataset

5.4 批大小设置

本次实验采用两组批大小：

- batch_size=16：作为 YOLO 风格小批量参考
- batch_size=256：作为大批量参考设置

5.5 控制变量

为保证对照严谨，正式 Stage2 实验固定如下参数：

- 测量目标样本数：4096
- 最少测量 batch 数：8
- `num_workers=8`
- `pin_memory=False`
- `persistent_workers=False`
- `prefetch_factor=4`
- `warmup_batches=3`
- `repeats=5`
- 运行顺序随机打散
- `device=cuda:0`

6 Stage1：加载器参数扫描

6.1 目的

在正式比较打包格式前，先单独扫描 `DataLoader` 参数，避免加载器设置掩盖存储格式差异。

6.2 结论

- `num_workers=8` 明显优于低 worker 设置
- `pin_memory` 并不总是提升性能
- `persistent_workers` 在部分大 batch 条件下有效
- 高并发设置会提高 RSS，需要在速度与资源占用之间平衡

6.3 结果图

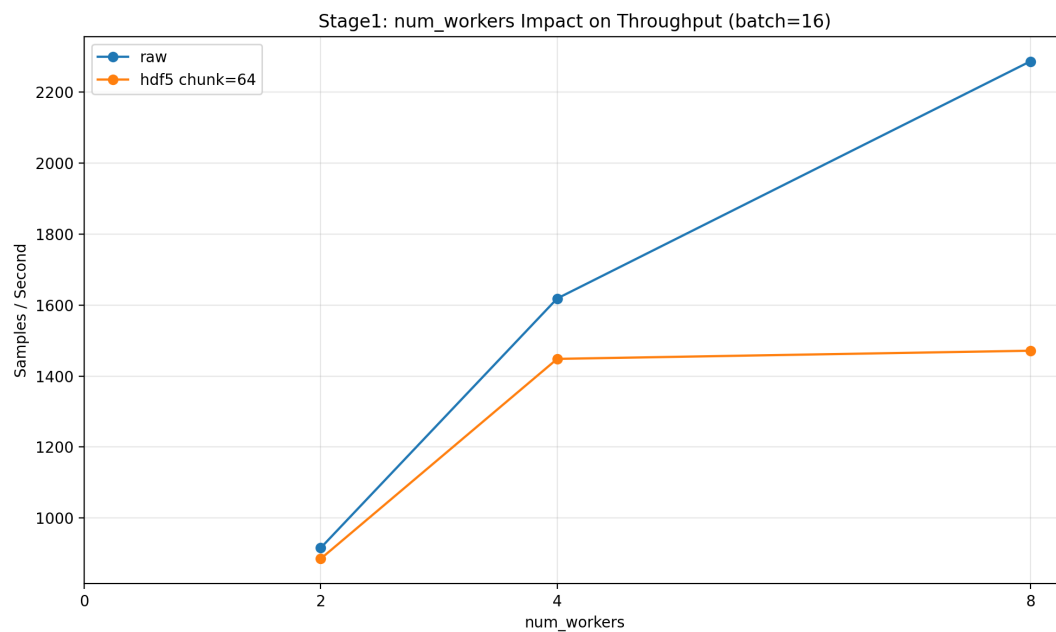


图 1: Stage1 中 batch_size=16 时 num_workers 对吞吐的影响

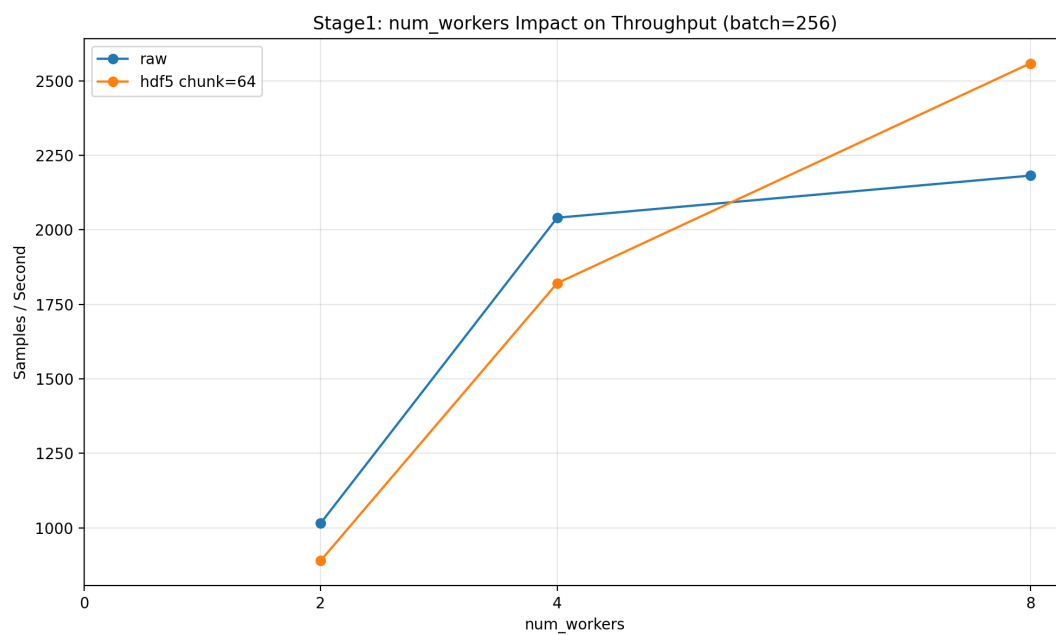


图 2: Stage1 中 batch_size=256 时 num_workers 对吞吐的影响

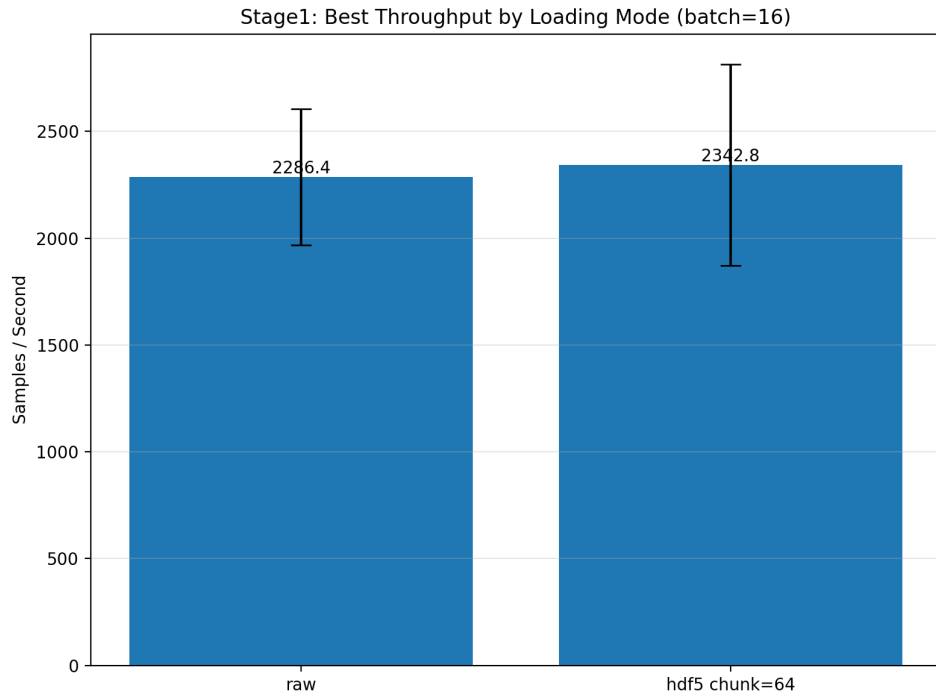


图 3: Stage1 中 batch_size=16 时 raw 与 chunked 的最佳加载器配置吞吐对比

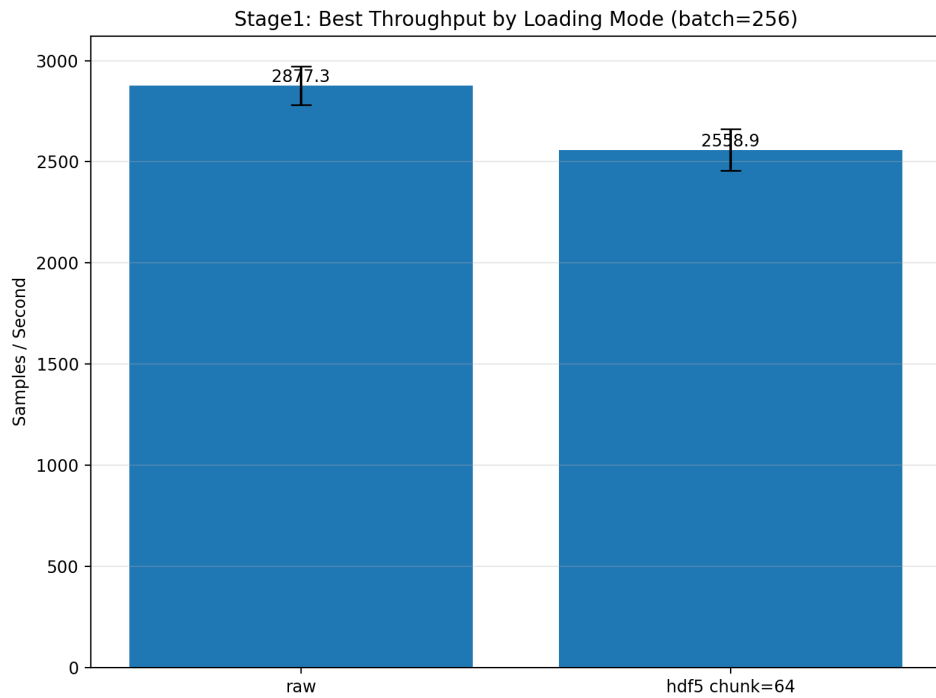


图 4: Stage1 中 batch_size=256 时 raw 与 chunked 的最佳加载器配置吞吐对比

6.4 关键结果

表 1: Stage1 最优加载器配置汇总

Batch Size	模式	Samples/s	num_workers	pin_memory	persistent_workers
16	raw	2286.357	8	False	False
16	hdf5 chunk=64	2342.771	8	True	False
256	raw	2877.332	8	False	True
256	hdf5 chunk=64	2558.852	8	False	False

Stage1 的作用不是直接寻找全局最快存储格式，而是确定一个足够合理的 DataLoader 基线。结果显示，`num_workers=8` 和 `prefetch_factor=4` 在当前实验范围内表现稳定，因此 Stage2 采用统一加载器基线继续比较不同打包格式与 chunk size。

7 Stage2: 格式与分块大小对比

7.1 实验设置

- chunk size: 32, 64, 128, 256, 512, 1024, 2048
- 格式: pickle, npz, pt, safetensors, hdf5, msgpack
- 基线: raw

7.2 结果图

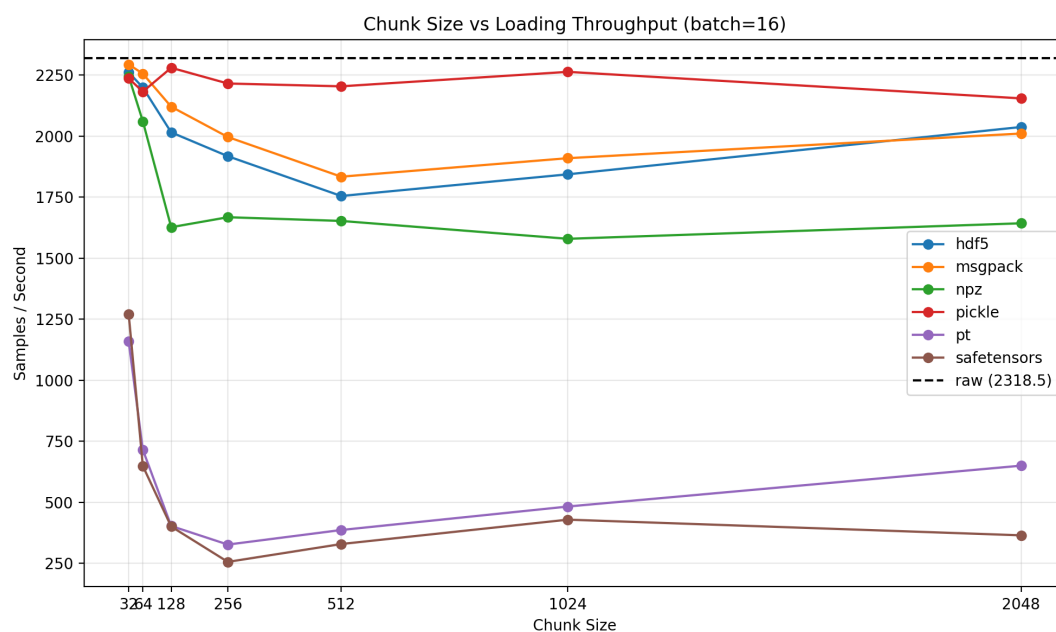


图 5: batch_size=16 时不同 chunk size 与格式的吞吐对比

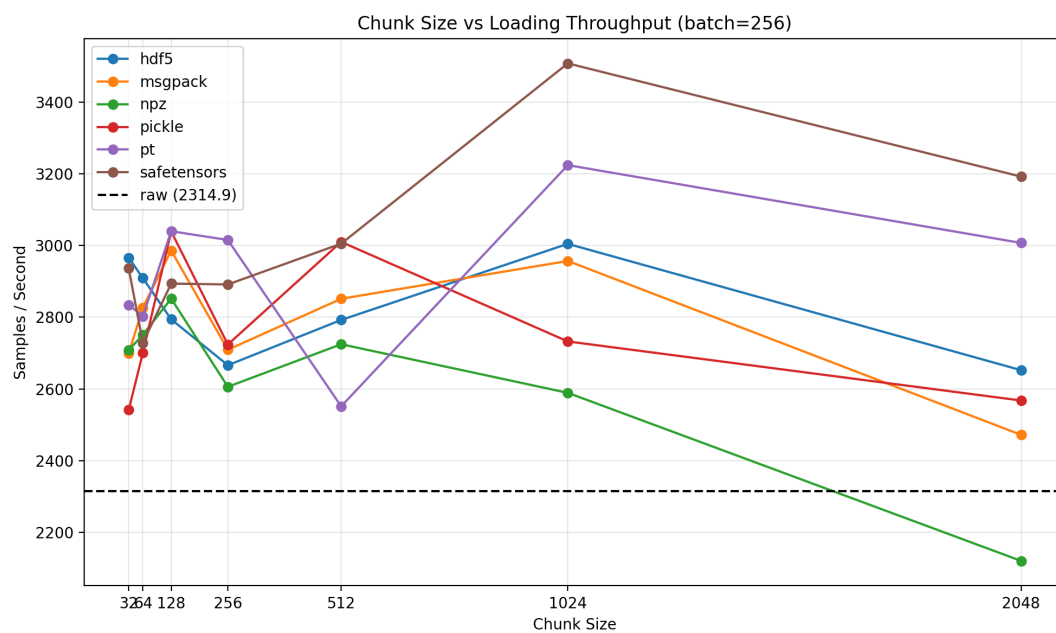


图 6: batch_size=256 时不同 chunk size 与格式的吞吐对比

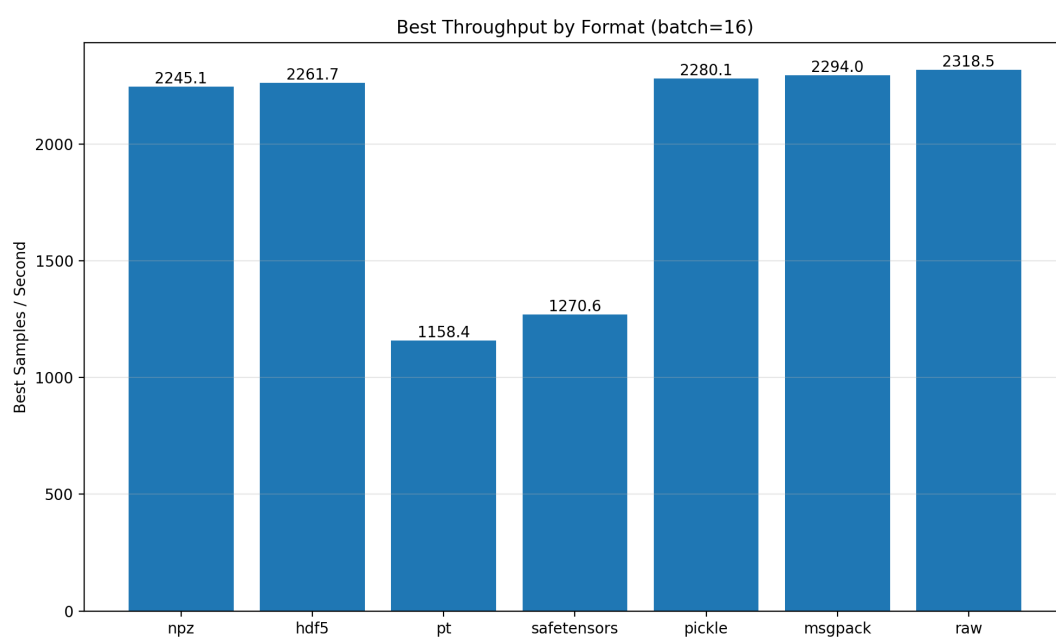


图 7: batch_size=16 时各格式最佳吞吐对比

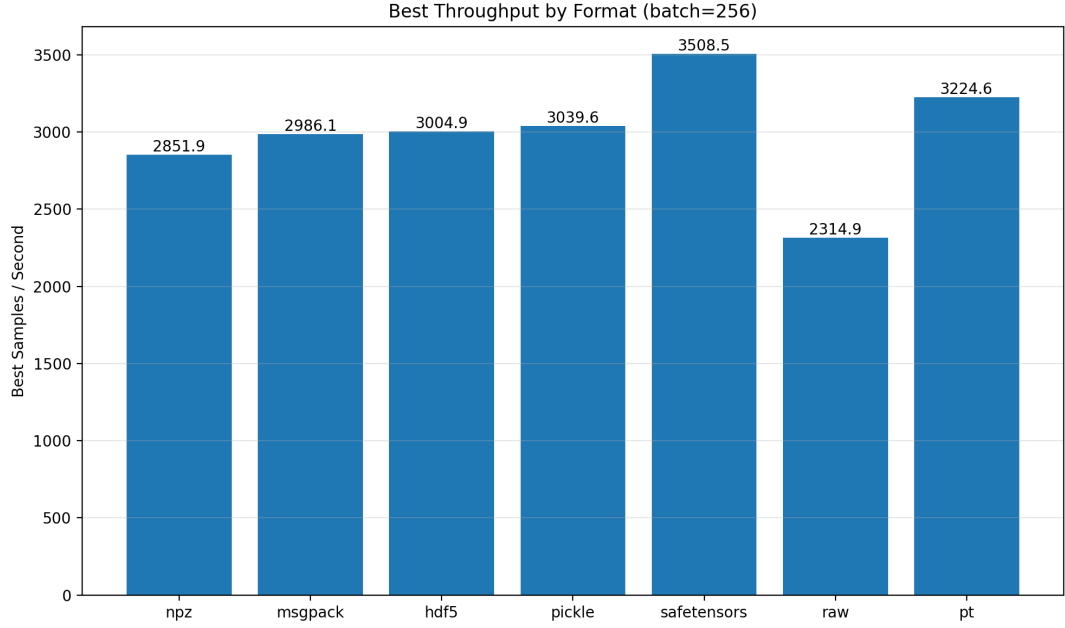


图 8: batch_size=256 时各格式最佳吞吐对比

7.3 关键结果

表 2: Stage2 最优结果汇总

Batch Size	最优方案	吞吐量 (samples/s)	Raw 基线 (samples/s)
16	raw	2318.536	2318.536
256	safetensors + chunk_size=1024	3508.513	2314.880

新的 Stage2 结果表明，小批量顺序读取时，直接读取统一 manifest 的 raw 路径仍然是最高吞吐方案；分块方法中，msgpack-32、pickle-128 与 hdf5-32 与 raw 的差距已经较小。大批量顺序读取时，safetensors-1024 达到最高吞吐，pt-1024、pickle-128 和 hdf5-1024 也保持了较高速度。

8 Stage3: shuffle 对照实验

8.1 实验目的

Stage2 的主要结论建立在 shuffle=False 的顺序读取条件下。为了评估更接近训练场景的随机打乱开销，额外设计 Stage3，对比 shuffle=False 与 shuffle=True 对吞吐的影响。

8.2 实验设置

- 数据规模：all_val 前 4096 个样本
- 代表格式：raw、safetensors、hdf5

- 代表 chunk size: 64、1024、2048
- batch size: 16、256
- 统一加载器基线: num_workers=8、pin_memory=False、persistent_workers=False、prefetch_factor=4
- 每组重复: 5 次

8.3 结果图

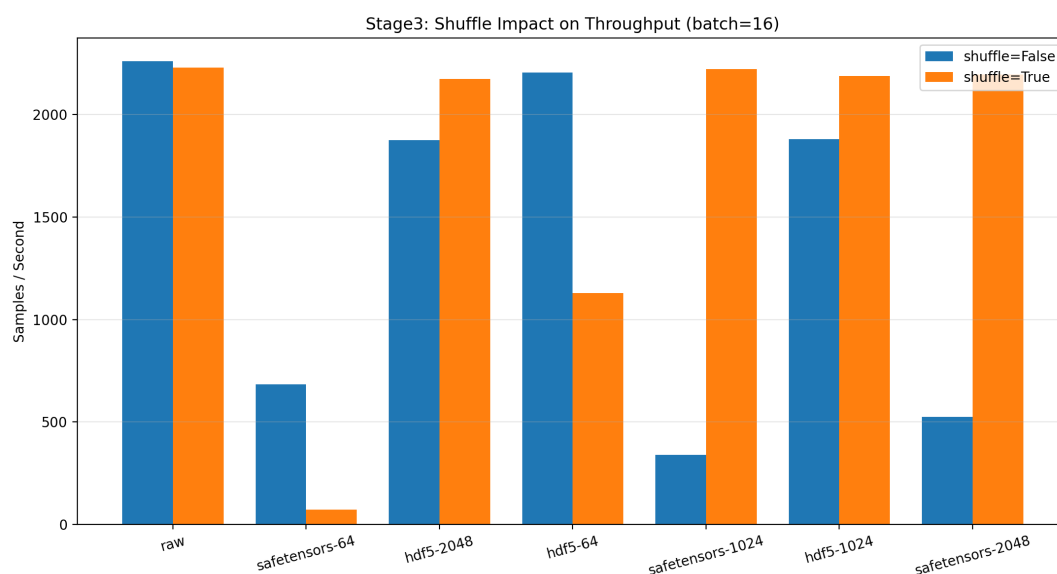


图 9: Stage3 中 batch_size=16 时开启/关闭 shuffle 的吞吐对比

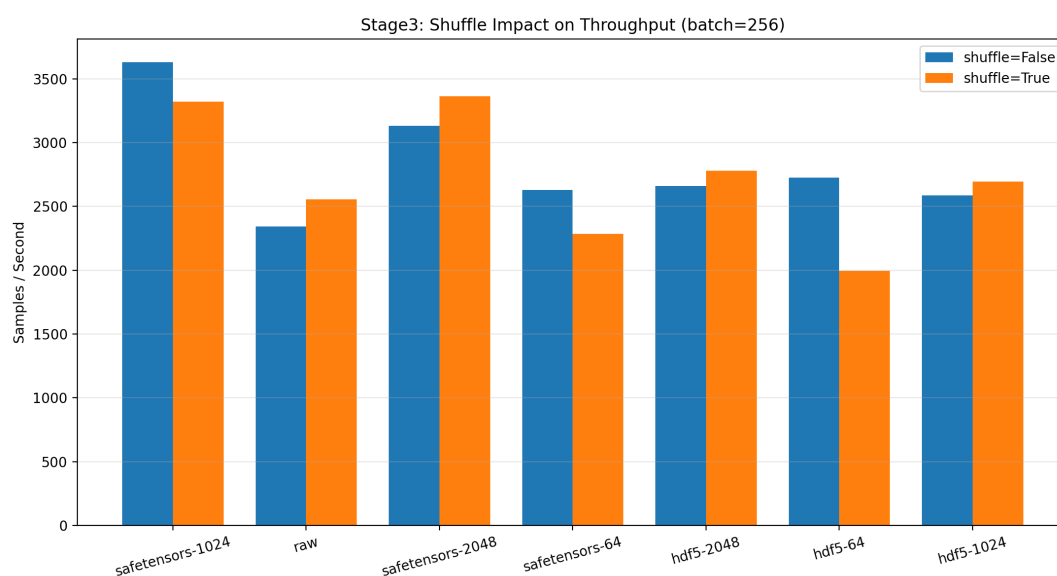


图 10: Stage3 中 batch_size=256 时开启/关闭 shuffle 的吞吐对比

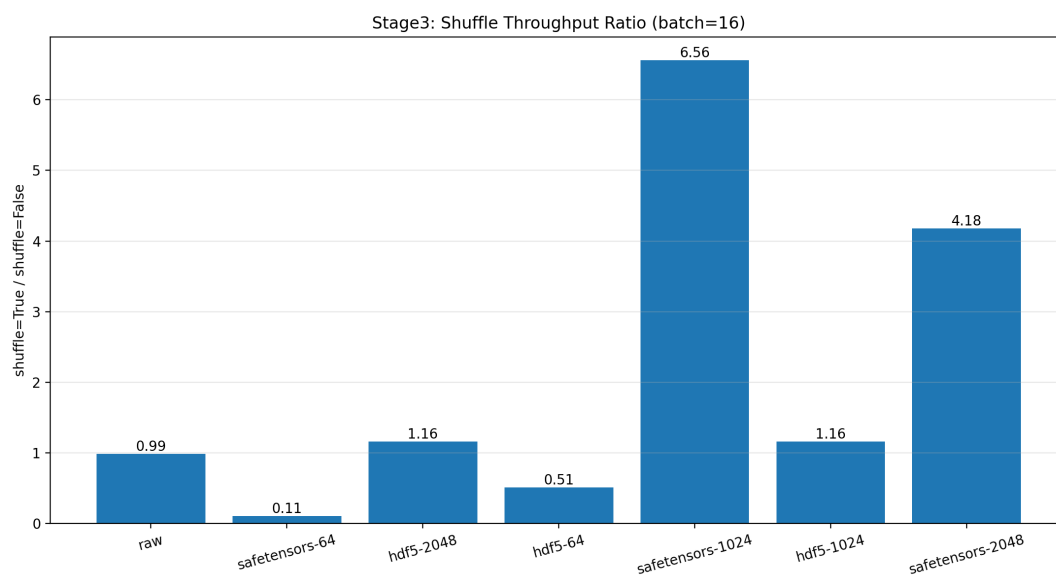


图 11: Stage3 中 batch_size=16 时 shuffle 惩罚比例

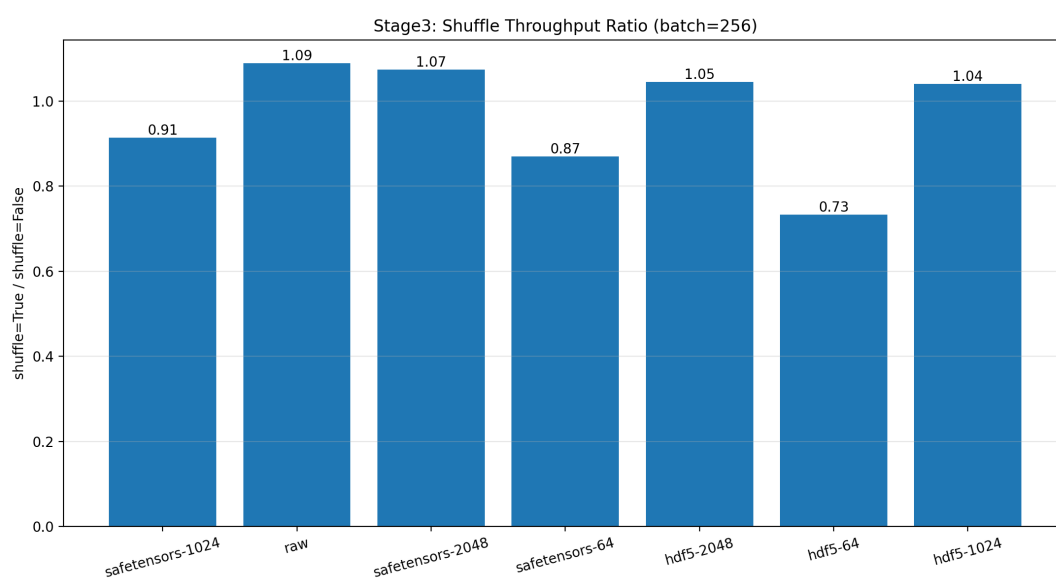


图 12: Stage3 中 batch_size=256 时 shuffle 惩罚比例

8.4 关键结果

表 3: Stage3 关键 shuffle 对照结果

Batch Size	配置	shuffle=False	shuffle=True	比值	说明
16	raw	2261.844	2230.617	0.986	基本稳定
16	safetensors-64	683.747	73.013	0.107	显著下降
16	safetensors-1024	338.648	2222.549	6.563	波动较大
16	safetensors-2048	525.461	2196.755	4.181	波动较大
16	hdf5-64	2205.509	1128.344	0.512	中等下降
16	hdf5-1024	1880.227	2188.617	1.164	基本稳定
256	raw	2344.712	2554.825	1.090	基本稳定
256	safetensors-64	2627.806	2286.791	0.870	轻度下降
256	safetensors-1024	3631.975	3321.697	0.915	基本稳定
256	safetensors-2048	3130.158	3362.538	1.074	基本稳定
256	hdf5-64	2726.197	1997.176	0.733	明显下降
256	hdf5-1024	2588.302	2694.242	1.041	基本稳定

Stage3 表明,在当前缓存策略与测量口径下, `shuffle=True` 并不会对所有配置都造成灾难性下降。两组 batch size 下, `raw` 都保持稳定; `batch_size=256` 时,多数中大块的 `safetensors` 与 `hdf5` 也保持在接近顺序读取的水平。更明显的 shuffle 惩罚主要出现在 `batch_size=16` 的 `safetensors-64` 和 `hdf5-64`。同时, `batch_size=16` 的部分中大块配置在 `shuffle=True` 下反而更高,说明这一规模下结果仍会受到缓存命中和调度节奏影响,应结合均值与标准差一起解释。

9 结论

本次实验在统一数据格式、统一 DataLoader 控制条件之后,完成了三个层次的对照: Stage1 用于确定加载器参数基线, Stage2 用于比较不同分块打包格式与 chunk size, Stage3 用于评估 shuffle 带来的随机读取开销。实验结果表明:

1. DataLoader 参数会显著影响吞吐,必须先单独扫描。当前实验范围内, `num_workers=8` 与 `prefetch_factor=4` 表现最稳定。
2. 分块打包方式会显著影响顺序读取吞吐, chunk size 也不能凭经验设定,必须通过扫描确定。
3. 在 `shuffle=False` 的顺序读取条件下,小批量场景最快方案为 `raw`,大批量场景最快方案为 `safetensors + chunk_size=1024`。
4. 在 `shuffle=True` 的随机打乱条件下, `raw` 与多数中大块配置整体保持稳定,但 `safetensors-64` 和 `hdf5-64` 在部分场景下会出现更明显的吞吐下降。

5. 若目标是大批量顺序读取的最高吞吐，可以优先采用 `raw`；若目标是大批量顺序读取，则 `safetensors + chunk_size=1024` 是当前实验中的最佳方案；若目标是兼顾随机打乱稳定性，则应优先考虑 `raw` 以及中大块的 `hdf5/safetensors` 方案。

10 附录：关键实现文件

- `build_unified_detection_manifest.py`
- `chunk_benchmark_lib.py`
- `pack_detection_chunks.py`
- `benchmark_detection_loading.py`
- `analyze_benchmark_results.py`
- `analyze_stage1_loader_scan.py`
- `analyze_stage3_shuffle.py`